

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Ilan Hawwari Prasojo – 5024241039

2026

1 Langkah-Langkah Percobaan

1.1 Blink pada STM32 Blackpill

Mengendalikan LED bawaan PC13 dengan Arduino dan mengunggah program melalui ST-Link.

1. Pastikan project Modul 1 sudah dibuat (Bagian D) dan isi `platformio.ini` sama seperti di atas

2. Hubungkan ST-Link ke Blackpill dengan kabel, lalu colokkan ST-Link ke komputer lewat USB. Cocokkan nama pinnya, karena letak pin ST-Link bisa berbeda-beda:

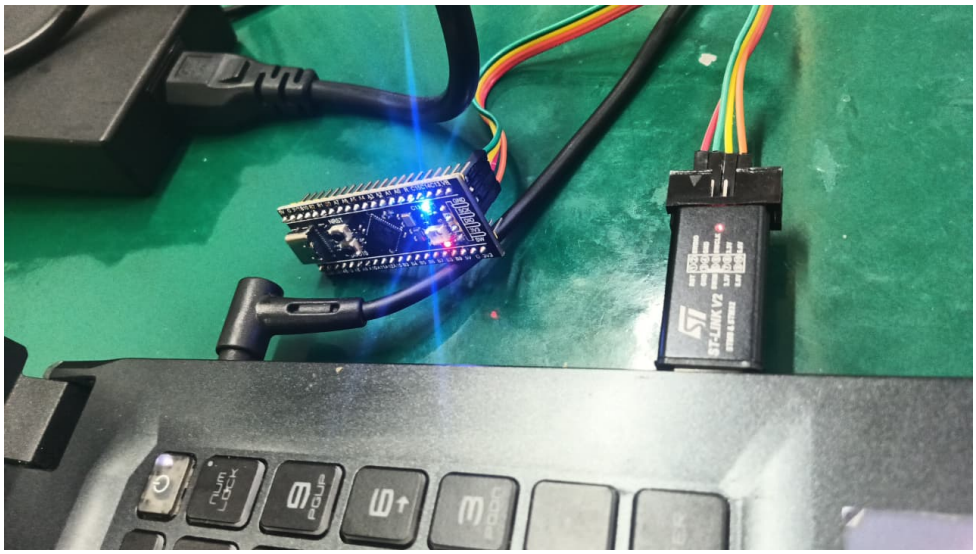
Pin ST-Link	Pin Blackpill	Fungsi
SWDIO	SWDIO	Jalur data/debug
SWCLK	SWCLK	Jalur clock/debug
GND	GND	Referensi ground bersama
3.3V	3V3	Referensi/tegangan target board

   Pastikan ST-Link memakai tegangan 3.3V.



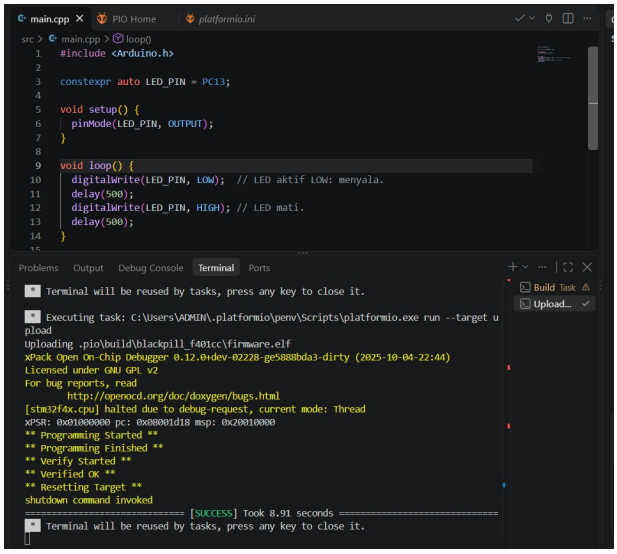
Adaptor ST-Link V2

Dokumentasi. STM32 Blackpill terhubung ke ST-Link V2 melalui kabel jumper



3. Tulis kode Blink di bawah pada `src/main.cpp`, lalu klik **Build** dan **Upload**

   Dokumentasi. Kode Blink PC13 dan log upload STM32 dengan status Verified OK



4. Lihat LED di board (PC13): nyala setengah detik, mati setengah detik, berulang terus

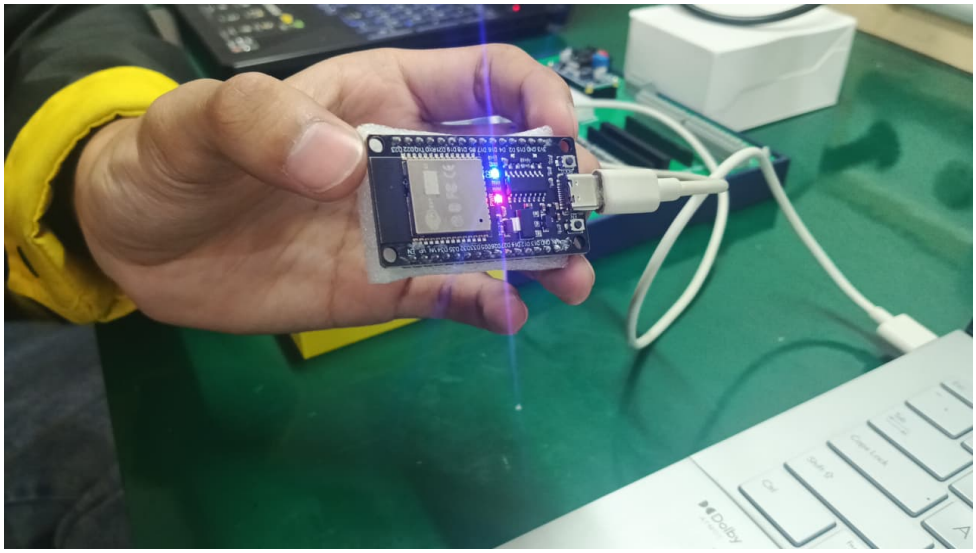
1.2 Blink pada ESP32 dengan ESP-IDF

Menjalankan Blink pada ESP32 dan membandingkan struktur programnya dengan Arduino.

1. Ganti isi `platformio.ini` dengan konfigurasi di atas (board ESP32 DevKit, framework ESP-IDF)

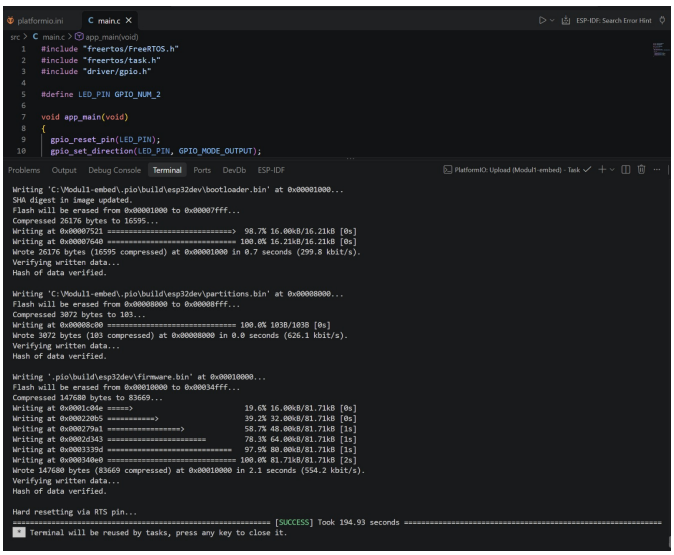
2. Colokkan ESP32 ke komputer lewat USB, lalu pastikan komputer mengenali port-nya

   Dokumentasi. ESP32 terhubung melalui USB dengan LED biru bawaan menyala



3. Hapus file `src/main.cpp`, buat file baru `src/main.c`, tulis kode Blink di bawah, lalu klik **Build** dan **Upload**

   Dokumentasi. Kode Blink ESP-IDF dan log upload ESP32 dengan status SUCCESS



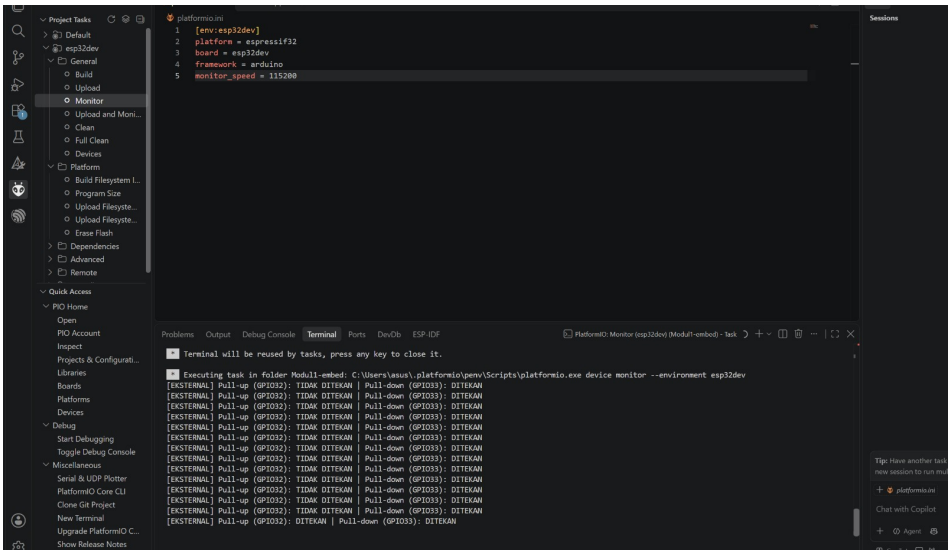
4. Analisis apa perbedaan memprogram dengan ESP-IDF dan dengan Arduino

1.3 Pull-up dan pull-down pada ESP32 (3A dan 3B)

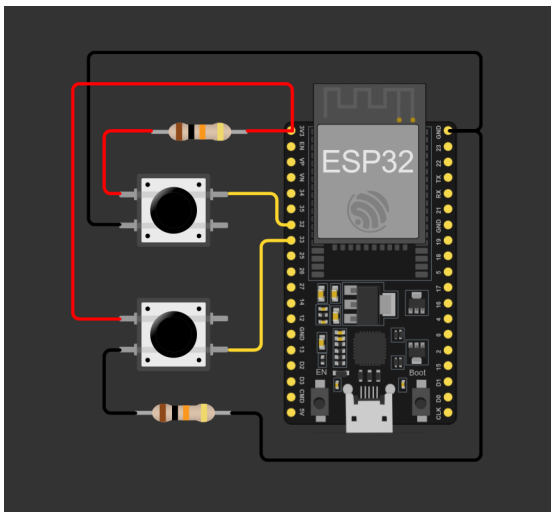
Membandingkan pembacaan dua tombol dengan resistor eksternal dan resistor internal pada GPIO32 dan GPIO33.

1. Ganti isi `platformio.ini` kembali ke konfigurasi Arduino di atas. Hapus `src/main.c`, lalu buat lagi file `src/main.cpp`

   Dokumentasi. Percobaan 3A, pengaturan ESP32 Arduino dan keluaran tombol eksternal

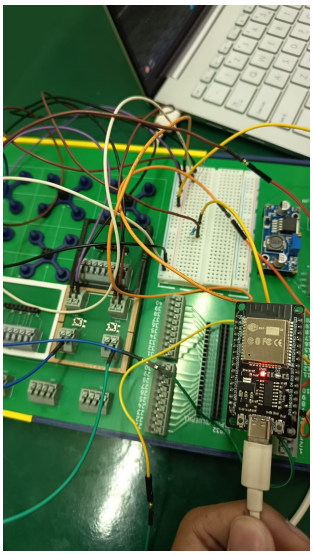


2. Pasang dua tombol di GPIO 32 dan 33 lengkap dengan resistornya, sesuai Skema Bagian A



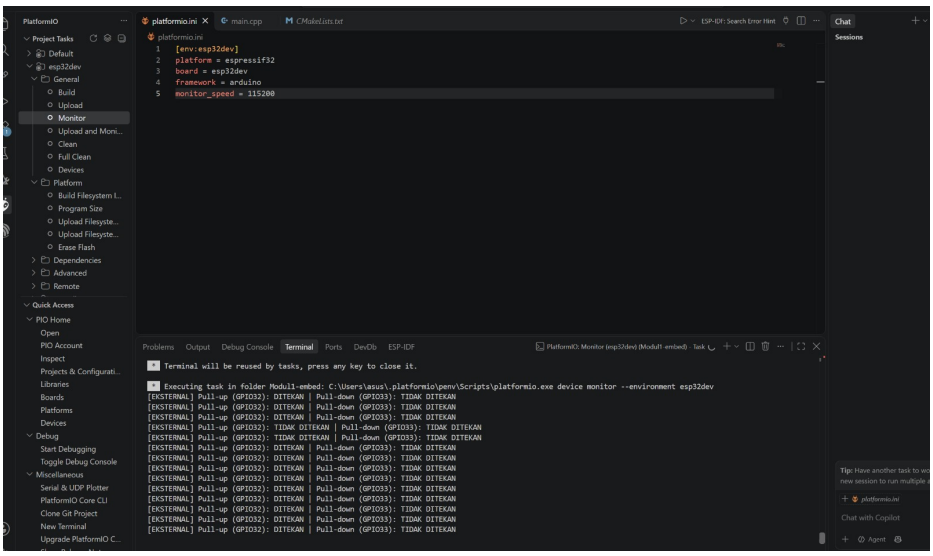
Gambar 6. Rangkaian pull-up dan pull-down eksternal

Dokumentasi. Rangkaian ESP32 dengan dua tombol dan resistor pada breadboard

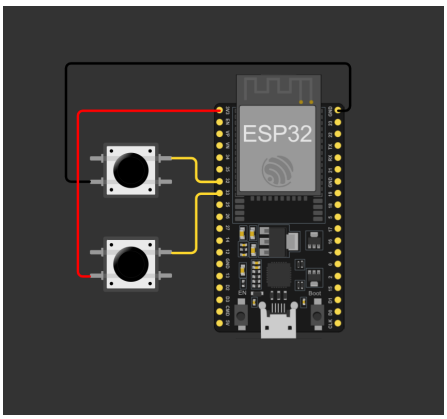


3. Tulis **Kode Program A (Eksternal)** di bawah pada `src/main.cpp`, klik Build dan Upload, lalu coba tekan kedua tombol dan catat hasilnya dari serial monitor.

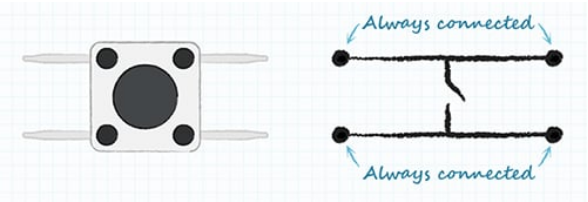
   Dokumentasi. Serial Monitor pada rangkaian eksternal saat status GPIO32 berubah



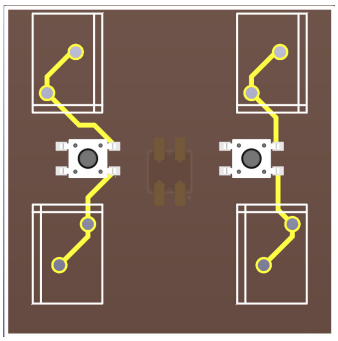
4. Lepas kedua resistor sesuai Skema Bagian B (Gambar 7), tombol tetap terpasang. Lihat Gambar 8 dan Gambar 9 untuk memastikan pasangan kaki tombol yang benar



Gambar 7. Rangkaian pull-up dan pull-down internal

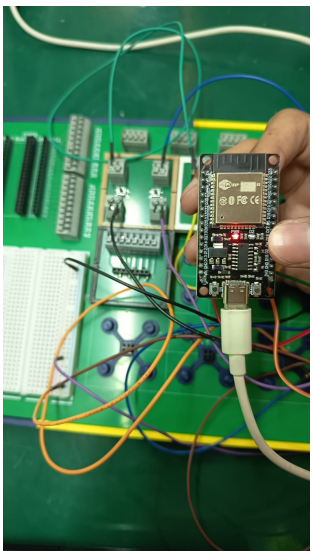


Gambar 8. Orientasi kaki pushbutton



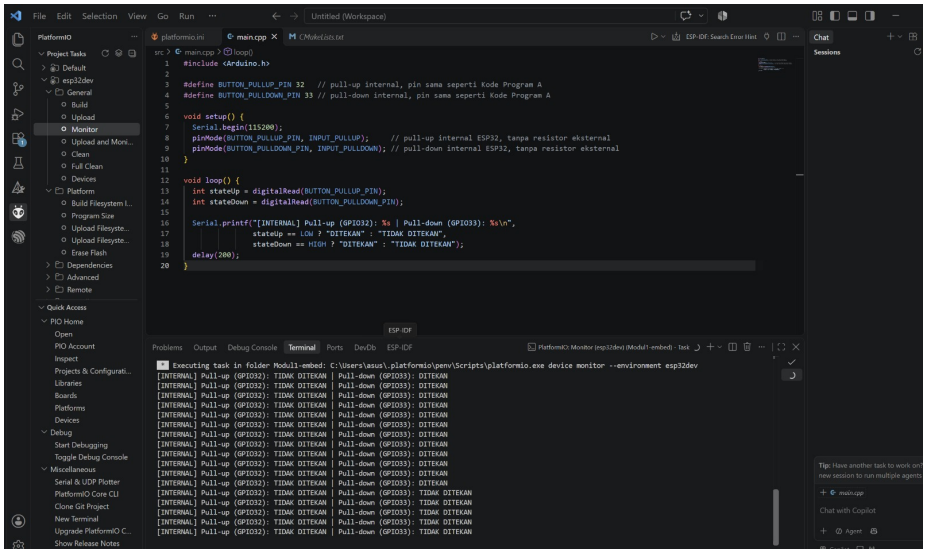
Gambar 9. Modul pushbutton

Dokumentasi. ESP32 dan sambungan ke dua tombol pada pengujian pull internal



5. Ganti isi `src/main.cpp` dengan **Kode Program B (Internal)** di bawah, klik Build dan Upload, lalu coba tombol yang sama dan catat hasilnya dari serial monitor.

   Dokumentasi. Percobaan 3B, kode pull internal dan perubahan status GPIO33



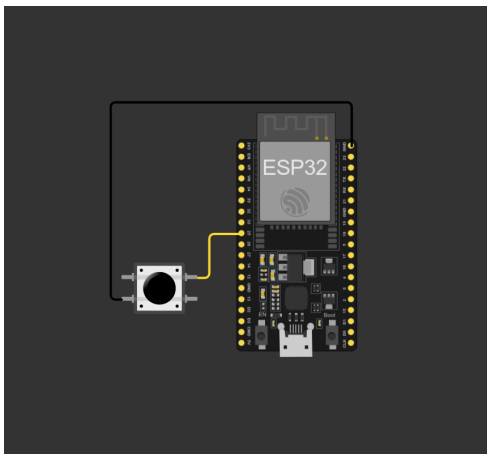
6. Bandingkan hasil A dan B. Seharusnya sama: pull-up bernilai HIGH saat tidak ditekan dan LOW saat ditekan, pull-down kebalikannya

7. Jika hasilnya terbalik atau berbeda, periksa apakah kedua resistor sudah benar-benar dilepas

1.4 Debouncing tombol

Membandingkan jumlah tekanan yang tercatat dengan dan tanpa penyaringan perubahan sinyal tombol.

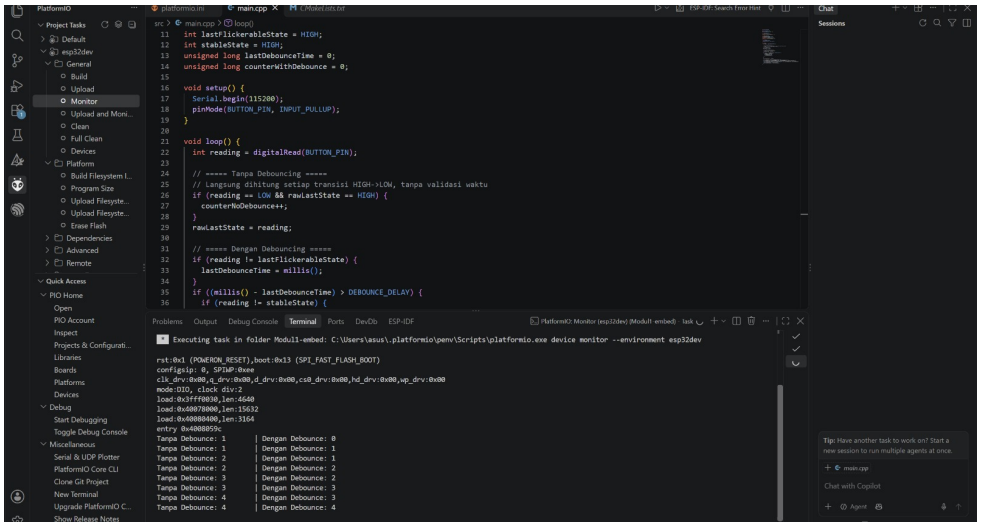
1. Pasang tombol sesuai skema di atas



Gambar 10. Rangkaian debounce pada GPIO25

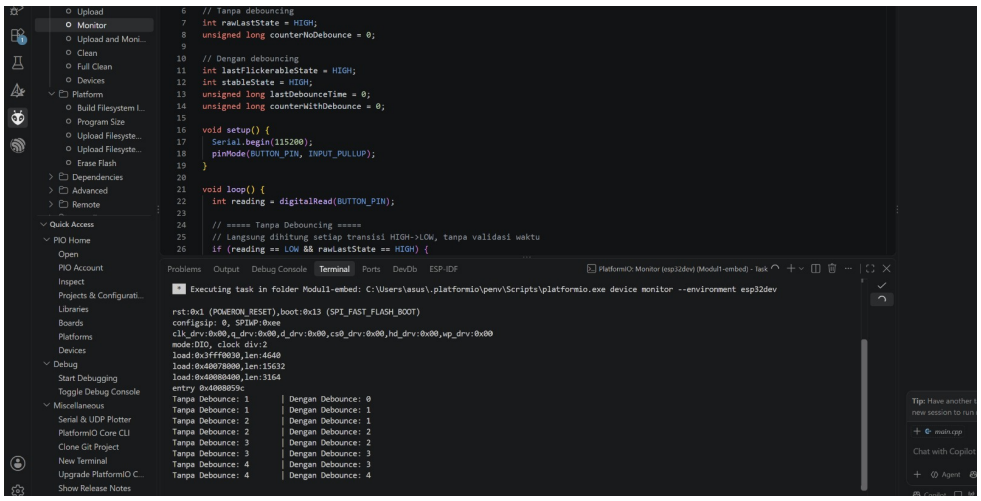
2. Tulis kode di bawah. Kode ini punya dua penghitung: `counterNoDebounce` (tanpa penyaring getaran tombol) dan `counterWithDebounce` (dengan penyaring)

   Dokumentasi. Bagian kode yang membandingkan penghitung tanpa dan dengan debounce



3. Tekan tombol satu kali, lalu bandingkan nilai kedua penghitung di Serial Monitor

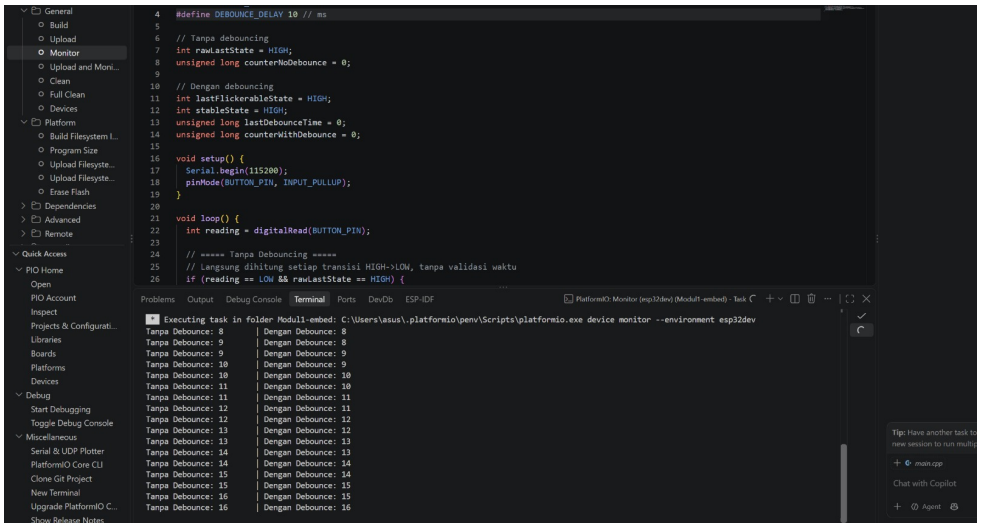
   Dokumentasi. Uji debounce 50 ms: kedua penghitung mencapai 4



4. Ulangi dengan cara menekan yang berbeda (cepat dan lambat). `counterNoDebounce` bisa naik lebih dari 1 untuk satu tekanan, sedangkan `counterWithDebounce` seharusnya naik tepat 1. Jika keduanya terlihat sama, coba tekan lagi karena getaran tombol tidak selalu muncul

5. Ubah `DEBOUNCE_DELAY` menjadi `10` (10 ms), klik Build dan Upload, lalu tekan tombol lagi

   Dokumentasi. Uji debounce 10 ms: kedua penghitung mencapai 16



6. Ubah menjadi `200` (200 ms), klik Build dan Upload, lalu tekan tombol dengan cepat berulang kali

   Dokumentasi. Uji debounce 200 ms: penghitung mentah mencapai 17, penghitung debounce tetap 0

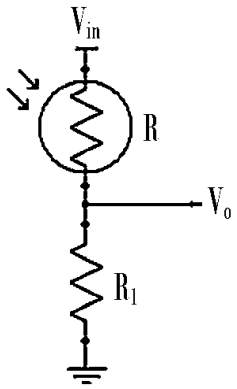


7. Bandingkan ketiga nilai (10, 50, dan 200 ms). Analisis untung ruginya: makin besar nilainya makin bersih dari getaran, tetapi tombol makin lambat merespons

1.5 Pembacaan ADC dengan LDR

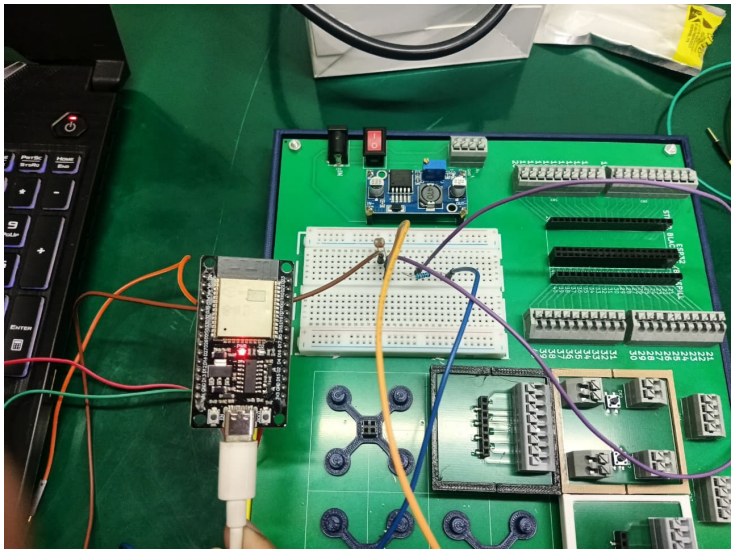
Mengamati perubahan nilai analog saat intensitas cahaya pada LDR berubah.

1. Pasang LDR bersama resistor sesuai skema, lalu hubungkan titik tengah keduanya ke GPIO 34



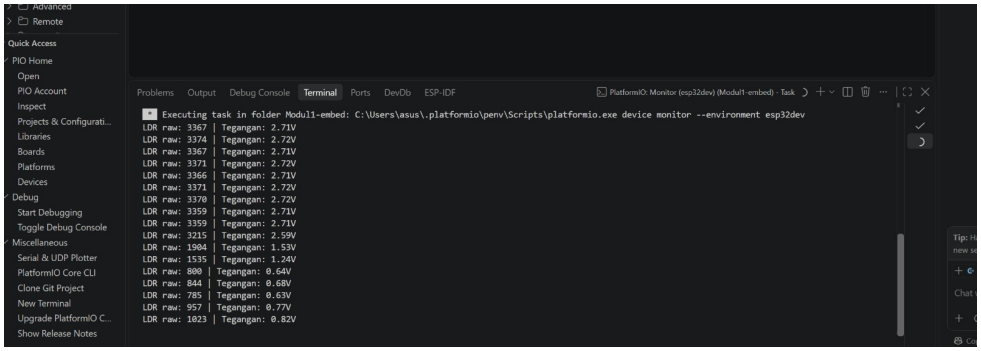
Gambar 11. Pembagi tegangan LDR pada GPIO34

Dokumentasi. LDR dan resistor pembagi tegangan pada breadboard



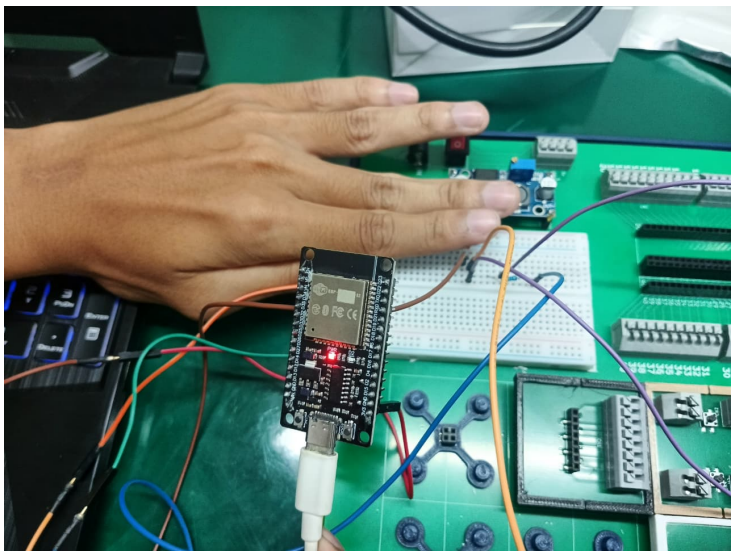
2. Tulis program yang membaca nilai ADC mentah dan mengubahnya menjadi tegangan
3. Klik **Build** dan **Upload**, lalu buka Serial Monitor

   Dokumentasi. Percobaan 5: pembacaan ADC LDR dan perkiraan tegangan pada Serial Monitor



4. Tutup LDR dengan tangan, lalu sinari dengan lampu. Lihat nilai mentahnya (0 sampai 4095) berubah

   Dokumentasi. Pengujian cahaya dengan tangan menutupi area LDR



5. Analisis mengapa sinyal analog bisa menunjukkan banyak kondisi, tidak hanya HIGH atau LOW

2 Analisis Hasil Percobaan

2.1 Blink pada STM32 Blackpill

LED bawaan pada PC13 aktif pada level LOW. Karena program memberi jeda 500 ms pada tiap level, satu siklus nyala dan mati diperkirakan berlangsung sekitar satu detik.

Saat upload ke Blackpill, log mencatat `Verified OK`. Setelah itu, OpenOCD me-reset board. LED biru pada Blackpill terlihat menyala setelah program diunggah.

2.2 Blink pada ESP32 dengan ESP-IDF

Program ESP-IDF masuk melalui `app_main()` dan mengatur arah GPIO lewat `gpio_set_direction()`. Selama `vTaskDelay()`, task Blink menunggu dan scheduler dapat menjalankan task lain. Pada Arduino, inisialisasi ditempatkan di `setup()` dan bagian yang berulang di `loop()`. Kedua program mengubah level keluaran setiap 500 ms dengan fungsi yang berbeda.

Saat upload ESP-IDF, saya mendapat pesan `Hash of data verified` dan `SUCCESS`. LED biru pada ESP32 terlihat menyala saat board tersambung ke laptop lewat USB.

2.3 Pull-up dan pull-down

Saat tombol pull-up dilepas, resistor menarik GPIO32 ke 3,3 V. Tombol yang ditekan menghubungkan pin ke GND sehingga nilainya LOW. Pada GPIO33, resistor pull-down menahan pin pada LOW saat tombol dilepas, sedangkan tekanan tombol menghubungkannya ke 3,3 V. Tabel berikut menunjukkan logika yang dipakai pada uji eksternal dan internal. Kedua cara memberi level awal yang sama pada pin. Pada uji resistor eksternal, saya melihat status kedua tombol berubah di Serial Monitor. Setelah beralih ke resistor internal, perubahan status GPIO33 juga muncul dengan logika yang sama.

Konfigurasi	Tombol dilepas	Tombol ditekan
Pull-up GPIO32	HIGH	LOW
Pull-down GPIO33	LOW	HIGH

2.4 Debouncing tombol

Kontak tombol bisa memantul beberapa kali sebelum stabil. Penghitung langsung menambah nilai pada setiap transisi HIGH ke LOW, sedangkan penghitung debounce menunggu level baru bertahan selama interval yang ditentukan. Pada 10 ms, tombol merespons lebih cepat, tetapi pantulan yang lebih panjang masih mungkin ikut terbaca. Pada 200 ms, penekanan atau pelepasan yang terlalu singkat dapat terlewat. Pada uji 10 ms, kedua penghitung mencapai 16. Pada uji 200 ms, penghitung tanpa debounce mencapai 17 sementara penghitung dengan debounce tetap 0. Pada interval ini, penghitung dengan debounce tidak bertambah karena program belum mendapat kondisi LOW yang cukup lama. Pada uji 50 ms, penghitung dengan debounce sempat tertinggal satu hitungan setelah perubahan terbaca, lalu menyusul setelah sinyal stabil. Keduanya mencapai 4 pada akhir log.

Debounce	Tanpa debounce	Dengan debounce	Catatan respons
10 ms	16	16	Kedua penghitung mencapai angka yang sama
50 ms	4 (nilai akhir)	4 (nilai akhir)	Penghitung debounce menyusul setelah sinyal stabil
200 ms	17 (nilai akhir)	0 (nilai akhir)	Tidak ada transisi stabil yang tercatat

2.5 Pembacaan ADC dengan LDR

Saat LDR mendapat lebih banyak cahaya, resistansinya cenderung turun. Karena LDR berada di sisi 3,3 V pada pembagi tegangan, tegangan di GPIO34 cenderung naik, diikuti kenaikan nilai ADC. ADC 12 bit menghasilkan kode 0–4095. Konversi dari modul memberi perkiraan tegangan karena hubungan kode ADC dan tegangan pada perangkat nyata dipengaruhi karakteristik ADC. Di Serial Monitor, pembacaan awal berada di kisaran 3359–3374 (2,71–2,72 V), lalu turun melalui 1904 (1,53 V) dan 1535 (1,24 V) hingga mencapai 785 (0,63 V). Saat pengujian, saya menutup area LDR dengan tangan untuk mengurangi cahaya yang masuk. Penurunan nilai ADC sesuai dengan cara kerja pembagi tegangan pada rangkaian ini.

Bagian log	Nilai ADC mentah	Tegangan perkiraan (V)
Awal	3359–3374	2,71–2,72
Saat turun	1904 dan 1535	1,53 dan 1,24
Nilai terendah	785	0,63

3 Hasil Tugas Modul

3.1 Rancangan matriks tombol dan LED 3 × 3

Saya menyusun sembilan tombol dalam tiga baris dan tiga kolom. Untuk membaca tombol, saya buat program yang menarik satu baris ke LOW, membaca ketiga kolom, lalu mengembalikan baris itu ke mode input sebelum pindah ke baris berikutnya. Saya menyimpan waktu perubahan dan status masing-masing tombol secara terpisah. Kalau pembacaan baru bertahan 50 ms, LED di posisi yang sama baru berubah. LED menyala selama tombol ditekan dan padam setelah pelepasan terbaca stabil.

Jalur	1	2	3
Baris tombol	GPIO25	GPIO26	GPIO27
Kolom tombol	GPIO32	GPIO33	GPIO4
LED baris 1	GPIO13	GPIO14	GPIO16
LED baris 2	GPIO17	GPIO18	GPIO19
LED baris 3	GPIO21	GPIO22	GPIO23

Saya memasang satu resistor 220 Ω untuk setiap LED dan menghubungkan katodanya ke GND. Saya menguji tombol satu per satu karena matriks tombol ini tidak memakai dioda.

3.2 Kode program ESP32 Arduino

Listing 1: Pemindaian tombol dan debounce per posisi

```
#include <Arduino.h>

const uint8_t rowPins[3] = {25, 26, 27};
const uint8_t colPins[3] = {32, 33, 4};
const uint8_t ledPins[3][3] = {
  {13, 14, 16}, {17, 18, 19}, {21, 22, 23}
};
const unsigned long debounceMs = 50;

struct ButtonState {
  bool raw;
  bool stable;
  unsigned long changedAt;
};
ButtonState buttons[3][3] = {};

void setup() {
  Serial.begin(115200);
  for (uint8_t row = 0; row < 3; ++row) {
    pinMode(rowPins[row], INPUT);
    pinMode(colPins[row], INPUT_PULLUP);
    for (uint8_t col = 0; col < 3; ++col) {
      pinMode(ledPins[row][col], OUTPUT);
      digitalWrite(ledPins[row][col], LOW);
    }
  }
}

void loop() {
  for (uint8_t row = 0; row < 3; ++row) {
    digitalWrite(rowPins[row], LOW);
    pinMode(rowPins[row], OUTPUT);
    delayMicroseconds(10);

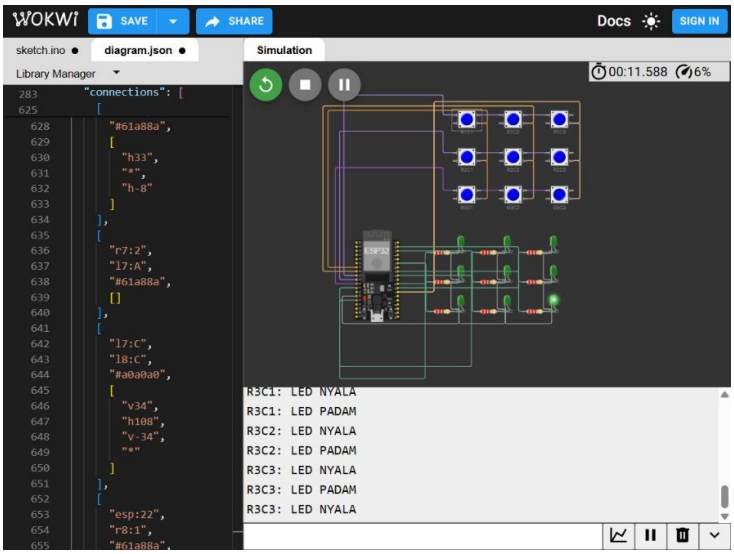
    for (uint8_t col = 0; col < 3; ++col) {
      auto &button = buttons[row][col];
      const bool pressed = digitalRead(colPins[col]) == LOW;
      const unsigned long now = millis();
      if (pressed != button.raw) {
```

```
        button.raw = pressed;
        button.changedAt = now;
      }
      if (button.raw != button.stable &&
          now - button.changedAt >= debounceMs) {
        button.stable = button.raw;
        digitalWrite(ledPins[row][col], button.stable ? HIGH : LOW);
        Serial.printf("R%uC%u: LED %s\n", unsigned(row + 1),
                      unsigned(col + 1), button.stable ? "NYALA" : "PADAM");
      }
    }
    pinMode(rowPins[row], INPUT);
  }
  delay(1);
}
```

3.3 Uji dan dokumentasi simulasi

Saya menguji rangkaian di Wokwi dengan debounce 50 ms per tombol. Setelah merapikan kabel, saya ulangi pengujian dari R1C1 sampai R3C3. Di setiap posisi saya mendapat satu pesan `LED NYALA` setelah tombol ditekan dan satu pesan `LED PADAM` setelah dilepas. Kode yang diuji menampilkan koordinat setiap kali status LED berubah.

Dokumentasi. Wiring terbaru di Wokwi saat tombol R3C3 ditahan dan LED R3C3 menyala



Kesembilan posisi menghasilkan pasangan NYALA dan PADAM di Serial Monitor.

3.4 Jawaban pertanyaan analisis

1. **Debouncing ketika hanya satu tombol ditekan.** Saya tetap memakai debounce walaupun tombol diuji satu per satu. Kontak pada satu tombol saja bisa memantul saat ditekan atau dilepas. Satu tekanan bisa menghasilkan beberapa perubahan HIGH dan LOW di kolom yang sama. Penyaringan selama 50 ms membuat LED mengikuti perubahan setelah kontak stabil, sehingga perubahan singkat selama pantulan tidak langsung mengubah LED.

2. **Pull-up pada kolom tombol.** Saya memakai pull-up supaya kolom terbaca HIGH saat tombol dilepas. Di kode, saya mengaktifkannya dengan `INPUT_PULLUP`. Saat tombol ditekan pada baris yang sedang aktif, kolom tersambung ke baris LOW. Jika kolom dibiarkan mengambang, gangguan kecil atau muatan pada pin dapat menggeser tegangan melewati ambang logika dan terbaca sebagai tekanan palsu.

3. **Input digital dan ADC.** Pembacaan digital membandingkan tegangan dengan ambang logika sehingga program menerima HIGH atau LOW. LDR dalam pembagi tegangan menghasilkan tegangan yang berubah mengikuti cahaya. ADC mengubah tegangan itu menjadi salah satu dari 4096 kode pada resolusi 12 bit, sehingga perubahan cahaya dapat dibedakan dalam banyak tingkat. Kalau satu tombol diganti LDR, saya akan memasang LDR dengan pembagi tegangan dan membaca titik tengahnya lewat GPIO34.

4 Kesimpulan

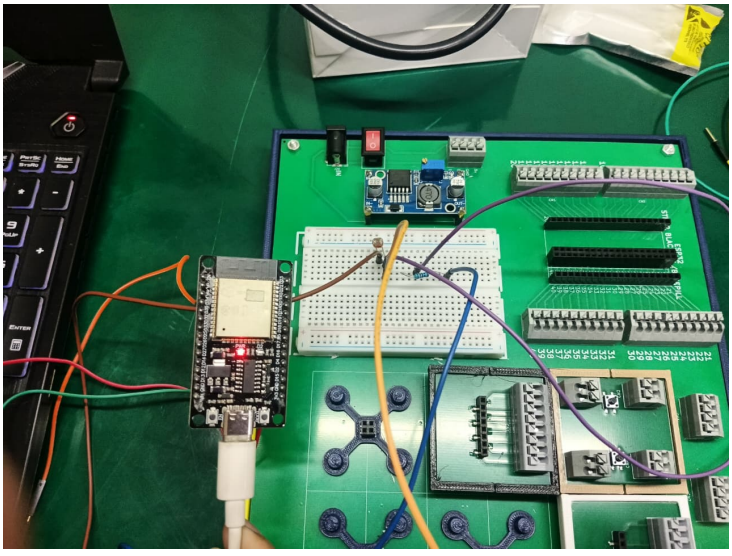
Di awal praktikum, saya mencoba program Blink di Blackpill dan ESP32. Keduanya sama-sama menyalakan LED, tetapi cara mengatur pin dan jeda di Arduino dan ESP-IDF berbeda. Saat pindah ke percobaan tombol, saya melihat bahwa pull-up dan pull-down memberi nilai awal yang berbeda ketika tombol dilepas. Di percobaan LDR, nilai ADC bisa turun bertahap ketika cahaya berkurang. Perubahan seperti ini tidak terlihat kalau pin hanya dibaca sebagai HIGH atau LOW.

Perbedaan waktu debounce paling terasa pada uji 200 ms. Penghitung langsung mencapai 17, sementara penghitung dengan debounce masih 0. Dari situ saya paham kenapa jedanya perlu disesuaikan dengan cara tombol ditekan. Di simulasi Wokwi, saya memakai 50 ms untuk setiap tombol. Saat kesembilan tombol dicoba bergantian, LED di posisi yang sama menyala ketika tombol ditahan dan padam setelah dilepas.

5 Lampiran

5.1 Dokumentasi saat praktikum

Dokumentasi. ESP32, breadboard, dan rangkaian LDR di meja praktikum



Dokumentasi. Pengujian LDR saat area sensor ditutup dengan tangan

